If you actually want to import the modules or any of their contents, you will need to use one of the forms that we have mentioned above.

Package Initialization

Packages can contain several submodules within it

While you read through most of the documentation available for Python, you will see examples that state an _init_.py file should be present in the package directory while creating a package. While this statement held true once upon a time, it is not an absolute fact any more. While the very presence of an _init_.py signified to Python that a package was being defined, it was not very useful. This file could contain initialisation code or could even be empty, but it was mandatory for it to be present. Python 3.3 saw the introduction of Implicit Namespace Packages. These allow for packages to be created without the mandatory _init_.py file. These files can be present if and where package initialization is required, but the compulsion has been lifted, making it an important upgrade on Python 3.3 and newer versions.

All of this will hopefully help in better understanding how you can gain access to the functionality that is available in a number of third-party as well as built-in modules that are available in Python. Going forward, if you plan on developing your own application, creating modules and packages will be extremely helpful in organizing as well as modularising your code, which will eventually make maintenance and debugging extremely easy.